Lecture 11: Grouped Analysis & Combining Data Frames

Harris Coding Camp – Standard Track

Summer 2024

Today's Lecture

In this lecture, we will cover:

- `group_by()` to make analysis across different subgroups
- `X_join()` and other functions to combine data frames

Analyzing data across different subgroups using `group_by()`

Grouping data with `group_by()`

Often, we want to repeat the same analysis across different **subgroups**. We can automate that with `group_by()`. We will:

- create new columns with `group_by()` + `mutate()`
- `filter()` data with group specific matching criteria

Grouped `mutate`: differences

Use case: You want to work with **differences** between sales over time.

```
july_texas_housing_data <-
  housing_data %>%
    filter(month == 7) %>%
    select(city, year, sales)

# Grouped by city
differenced_data <-
  july_texas_housing_data %>%
    group_by(city) %>%
    mutate(last_year_sales = lag(sales),
           diff_sales = sales - last_year_sales)
```

Grouped `mutate`: differences

Use case: You want to work with **differences** between sales over time.[1]

```
differenced_data
```

```
#> # A tibble: 736 x 5
#> # Groups:   city [46]
#>    city     year sales last_year_sales diff_sales
#>    <chr>   <dbl> <dbl>           <dbl>      <dbl>
#>  1 Abilene  2000   152              NA         NA
#>  2 Abilene  2001   134             152        -18
#>  3 Abilene  2002   159             134         25
#>  4 Abilene  2003   171             159         12
#>  5 Abilene  2004   176             171          5
#>  6 Abilene  2005   177             176          1
#>  7 Abilene  2006   169             177         -8
#>  8 Abilene  2007   239             169         70
#>  9 Abilene  2008   171             239        -68
#> 10 Abilene  2009   158             171        -13
#> # i 726 more rows
```

[1] `lag()`'s sibling is `lead()` which will give you data from the following year.

Grouped `mutate`: ranking

Use case: You want to **rank** sales across cities in each year.

```
ranked_data <-
  july_texas_housing_data %>%
    group_by(year) %>%
    mutate(sales_rank = rank(-sales))
```

Grouped `mutate`: ranking

Use case: You want to **rank** sales across cities in each year.

```
ranked_data %>%
  arrange(year, sales_rank)
```

```
#> # A tibble: 736 x 4
#> # Groups:   year [16]
#>    city                year sales sales_rank
#>    <chr>              <dbl> <dbl>      <dbl>
#>  1 Houston             2000  5009          1
#>  2 Dallas              2000  4276          2
#>  3 Austin              2000  1818          3
#>  4 San Antonio         2000  1508          4
#>  5 Collin County       2000  1007          5
#>  6 Fort Bend           2000   753          6
#>  7 NE Tarrant County   2000   686          7
#>  8 Denton County       2000   638          8
#>  9 Fort Worth          2000   548          9
#> 10 Montgomery County   2000   463         10
#> # i 726 more rows
```

Ungrouping

To get rid of groups, use `ungroup()`:

```
annual_city_housing_prices %>%
  ungroup()
```

Try it yourself!

1. Using the `housing_data` dataset, filter observations where `city` is Brazoria County and group by `year`.
2. Next, use `summarize()` to create a variable that represents the total sales in each year.
3. Create a plot that represents the total sales over time.

Combining data frames horizontally using `X_join()`

Combining data frames horizontally using `X_join()`

- It's rare that a data analysis involves only a single data frame.
- Typically, you have many data frames, and you must **join** them together to answer the questions you're interested in.
- We will learn how to add new variables to one data frame from matching observations in another.

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

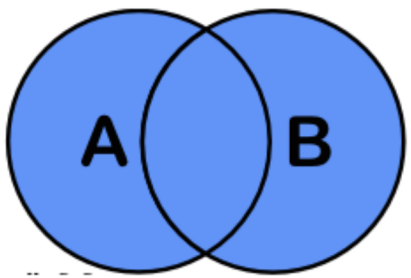
id	name	micro
1	Jeff	95
2	Regina	90
3	Sheng-Hao	80

After:

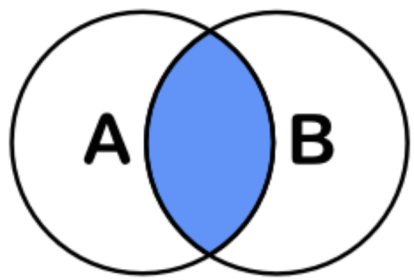
id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80

Types of horizontal join

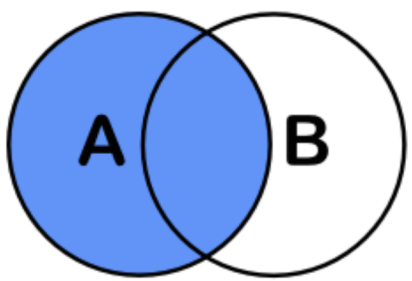
- full_join
- inner_join
- left_join
- right_join



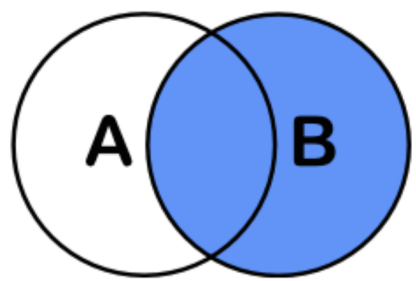
full_join(A,B)



inner_join(A,B)



left_join(A,B)



right_join(A,B)

inner_join

- Only keeps rows that occur in **both** data frames

```
inner_join(data1, data2, by = "id")
```

```
# This does the same
data1 %>%
  inner_join(data2, by = "id")
```

- by: the variable(s) to join by. By default, `join` will use the variable(s) with common names across the data frames.

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90

left_join

- Keeps all rows in the **first** data frame

```
left_join(data1, data2, by = "id")
```

```
# This does the same
data1 %>%
  left_join(data2, by = "id")
```

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	NA

right_join

- Keeps all rows in the **second** data frame

```
right_join(data1, data2, by = "id")
```

```
# This does the same
data1 %>%
  right_join(data2, by = "id")
```

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
4	Marley	NA	76

full_join

- Keeps all rows in **both** data frames

```
full_join(data1, data2, by = "id")
```

```
# This does the same
data1 %>%
  full_join(data2, by = "id")
```

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	NA
4	Marley	NA	76

join by a variable with different names

- Occasionally, we would like to join two data frames on **different column names**.

```
data1 %>%
  left_join(data2, by = "id")
```

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id_number	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

```
Error: Join columns must be present in data.
x Problem with `id`.
```

join by a variable with different names

- Let R know that id is the same as id_number.

```
data1 %>%
  left_join(data2, by = c("id" = "id_number"))
```

Before:

id	name	stats
1	Jeff	80
2	Regina	85
3	Sheng-Hao	90

id_number	name	micro
1	Jeff	95
2	Regina	90
4	Marley	76

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	NA

Try it Yourself!

- First, let's create two small datasets – Copy and run the code chunk below to assign these to addr and phone.

```
addr <- data.frame(name = c("Alice","Bob",
                            "Carol","Dave",
                            "Eve"),
                   email = c("alice@company.com",
                             "bob@company.com",
                             "carol@company.com",
                             "dave@company.com",
                             "eve@company.com"),
                   stringsAsFactors = FALSE)

phone <- data.frame(firstname = c("Bob","Carol",
                                  "Dave","Eve",
                                  "Frank"),
                    phone = c("919 555-1111",
                              "919 555-2222",
                              "919 555-3333",
                              "310 555-4444",
                              "919 555-5555"),
                    stringsAsFactors = FALSE)
```

Try it Yourself!

- How would you correctly **left join** these two datasets? What is the resulting data frame? Is there any missing value?
- Redo the above question using **right join**.
- Redo the above question using **inner join**.
- Redo the above question using **full join**.

Combining data frames vertically

Combining data frames vertically

- Sometimes, you have to **join** different data frames **vertically** in your project.
- We will learn how to add new rows to one data frame from matching criteria in another.

Before:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80

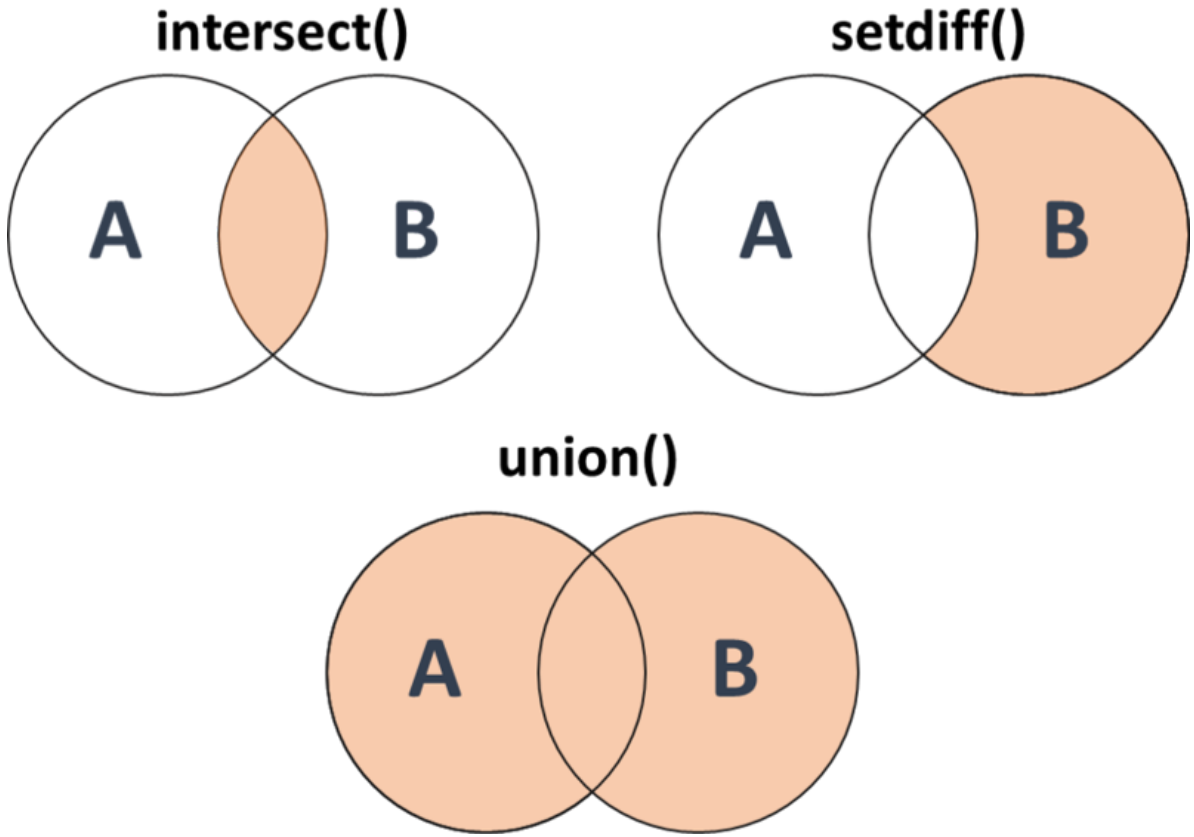
id	name	stats	micro
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

Types of vertical join

- union
- intersect
- setdiff



union

- Keeps rows that appear in **either** data frame.

```
union(data1, data2)
```

```
# This does the same
data1 %>%
  union(data2)
```

Before:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80

id	name	stats	micro
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

intersect

- Keeps rows that appear in **both** data frames.

```
intersect(data1, data2)
```

```
# This does the same
data1 %>%
  intersect(data2)
```

Before:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80

id	name	stats	micro
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

After:

id	name	stats	micro
3	Sheng-Hao	90	80

setdiff

- Keeps rows that appear in data1 but not data2.

```
setdiff(data1, data2)
```

```
# This does the same
data1 %>%
  setdiff(data2)
```

Before:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90
3	Sheng-Hao	90	80

id	name	stats	micro
3	Sheng-Hao	90	80
4	Marley	70	76
5	Hanna	100	100

After:

id	name	stats	micro
1	Jeff	80	95
2	Regina	85	90

Recap

This lesson gave you an idea about how to:

- `group_by()` to make analysis across different subgroups
- `X_join()` and other functions to combine data frames